CENTRO PAULA SOUZA
ETEC PROFESSOR CAMARGO ARANHA
TÉCNICO EM DESENVOLVIMENTO DE SISTEMAS

GROWLY

RELATÓRIO TÉCNICO E HISTÓRICO DE DESENVOLVIMENTO DO BANCO DE DADOS

MySQL → PostgreSQL / Supabase
Modelagem, arquitetura, implementação, segurança e evolução técnica

Equipe: Laís Silva Ferrari; Letícia Rodrigues Cardoso; Rebeca Carvalho Trindade; Thiago Nestor Afonso dos Santos.

São Paulo • Outubro de 2026

Apresentação e critérios de documentação

Este relatório reúne evidências do desenvolvimento do banco de dados do Growly, sistema voltado à identificação e ao conhecimento de Plantas Alimentícias Não Convencionais (PANCs). O texto integra o SQL histórico do protótipo MySQL, dez conjuntos de comandos SQL compartilhados para a versão PostgreSQL/Supabase, capturas do Table Editor, do Schema Visualizer e do Storage, e o Diário de Bordo Oficial da equipe.

Critério de evidência: “documentado” significa presente em script, imagem ou diário; “relatado” indica informação fornecida pela equipe; “recomendação” indica avaliação técnica, não implementação confirmada. Scripts representam diferentes momentos e não devem ser executados sequencialmente sem consolidação. Nenhuma execução de testes foi realizada diretamente neste relatório.

Resumo executivo

A versão documentada contém oito tabelas próprias no esquema public — admins, identificacoes, imagens, imagens_treinamento, jardim, mensagens, pancs e perfis — relacionadas à autenticação gerenciada em auth.users. Há dois buckets de armazenamento, avatares e imagens-treinamento, ambos com limite de 5 MB por arquivo. A arquitetura incorpora chaves estrangeiras, políticas de Row Level Security (RLS), trigger de criação automática de perfis e verificação de privilégios administrativos.

1. Contextualização e objetivos

O Growly surgiu de pesquisas acadêmicas sobre PANCs e de discussões de escopo do Trabalho de Conclusão de Curso. O banco de dados apoia o cadastro de usuários, a consulta ao catálogo de plantas, a manutenção do jardim pessoal, o registro de identificações, a associação de imagens, a interação com a assistente Eden e o fluxo de imagens de treinamento. A estrutura de dados foi alterada ao longo do projeto para acompanhar as necessidades do aplicativo.

1.1 Requisitos funcionais observáveis

- Registrar dados complementares do usuário autenticado, sem armazenar senhas próprias na tabela de perfis.
- Consultar catálogo de PANCs com nome, identificador textual, informações e conteúdo educativo.
- Permitir a cada usuário incluir, consultar, atualizar e remover itens do próprio jardim.
- Registrar identificações e seus resultados, com vínculo opcional a uma PANC catalogada.
- Associar imagens a usuários, espécies e identificações.
- Registrar perguntas e respostas relacionadas à assistente Eden.
- Receber imagens para possível treinamento, com autorização, status e revisão administrativa.

1.2 Requisitos não funcionais e regras de negócio

- Restringir dados pessoais por identidade autenticada.
- Preservar integridade referencial e registrar datas de criação.

- Limitar formatos e tamanho dos arquivos nos buckets configurados.
- Separar funções de usuários comuns e administradores.
- Evitar tratar resultado automatizado de identificação como garantia de comestibilidade ou segurança alimentar.

2. Histórico do desenvolvimento

Período	Evidência e marco
12/02/2026	Diário: primeira orientação formal do TCC e direcionamento de pesquisa para PANCs.
09/04/2026	Diário: elaboração do diagrama de caso de uso.
23/04/2026	Diário: registro de protótipo de banco de dados já finalizado e diagramas concluídos.
21/05/2026	Diário: revisão e organização do banco de dados.
A partir de 01/09/2026	Início da migração para Supabase/PostgreSQL, conforme relato da equipe.
Setembro–outubro/2026	Scripts e capturas documentam tabelas atuais, políticas RLS, administradores, buckets e ajustes no catálogo.

A equipe estima que o trabalho inicial em MySQL começou após 15 de março de 2026. A data exata não está comprovada pelo diário; a evidência documental mais segura é que havia um protótipo concluído em 23 de abril. A migração foi iniciada a partir de 1º de setembro, sem que isso implique conclusão nessa data.

3. Modelagem original em MySQL

O arquivo “GROWLY DATABASE.sql” contém sete tabelas: USUARIO, PANC, IDENTIFICACAO, IMAGEM, JARDIM, JARDIM_PANC e MENSAGEM. O script usa comandos típicos do MySQL, incluindo DROP DATABASE, CREATE DATABASE e USE. A separação entre JARDIM e JARDIM_PANC representa a modelagem inicial da coleção de plantas de cada usuário.

3.1 Modelo conceitual

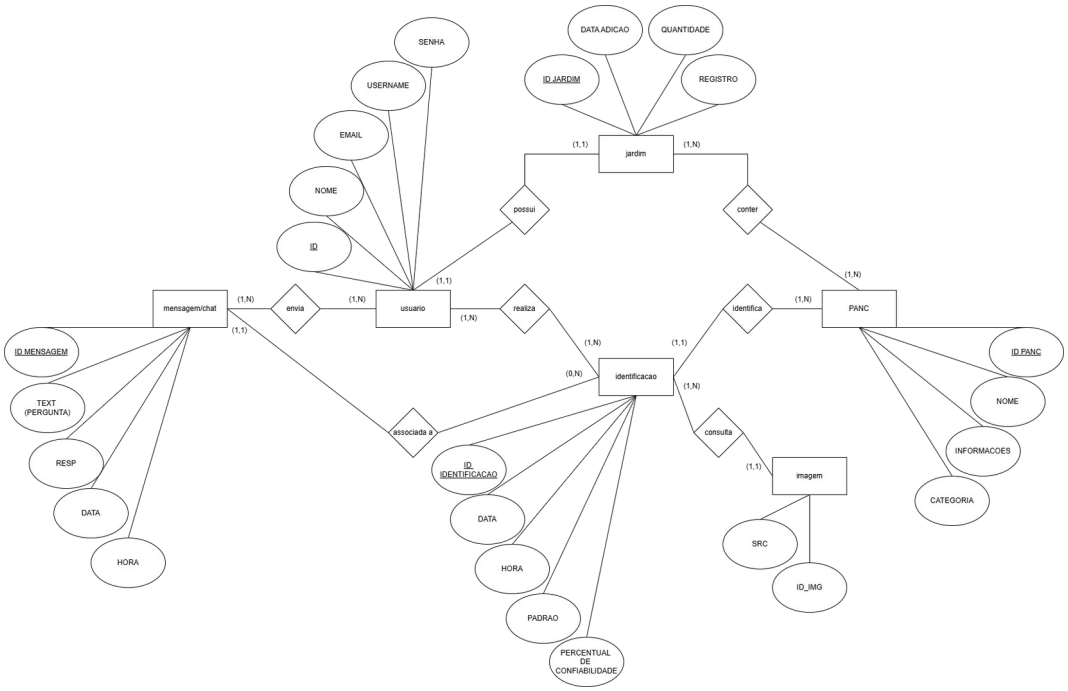


Figura — Modelo conceitual do protótipo inicial.

3.2 Modelo lógico

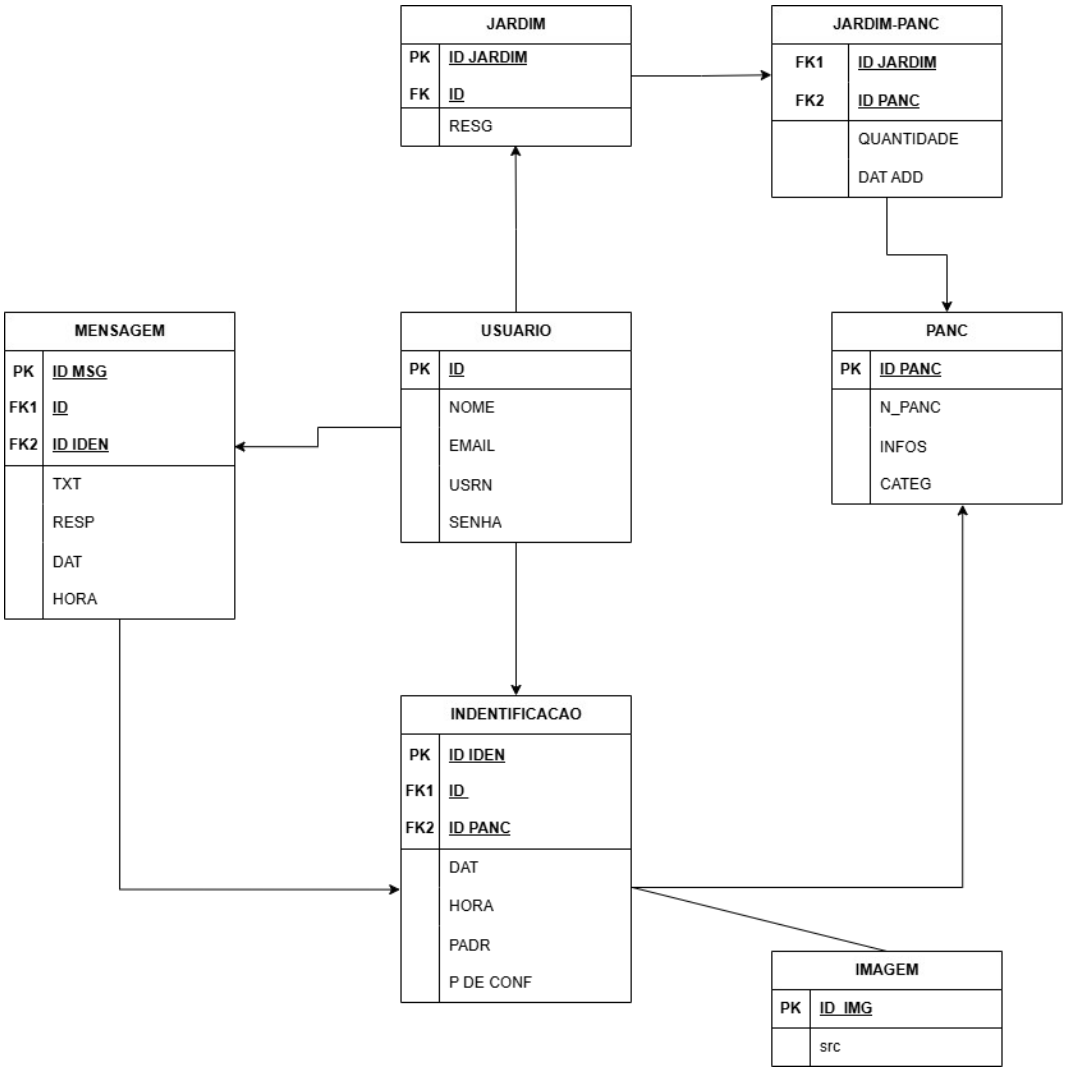


Figura — Modelo lógico do protótipo inicial.

3.3 Modelo físico

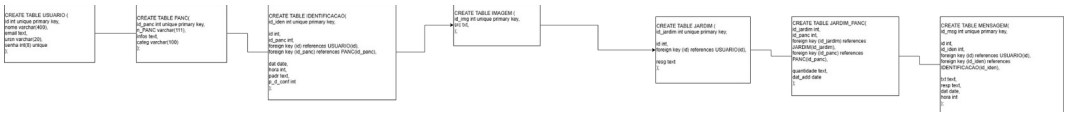


Figura — Modelo físico do protótipo inicial.

3.4 Dicionário sintético do protótipo

Tabela	Campos observados	Papel
USUARIO	id, nome, email, ursn, senha	Dados e credenciais no modelo inicial
PANC	id_panc, n_PANC, infos, categ	Catálogo de plantas
IDENTIFICACAO	id_iden, id, id_panc, dat, hora,	Resultados de identificação

	padr, p_d_conf	
IMAGEM	id_img, src	Referência a imagem
JARDIM	id_jardim, id, resg	Jardim vinculado ao usuário
JARDIM_PANC	id_jardim, id_panc, quantidade, dat_add	Associação jardim–PANC
MENSAGEM	id_msg, id, id_iden, txt, resp, dat, hora	Histórico de mensagens

3.5 Limitações técnicas históricas

O protótipo previa “senha int(8) unique”, solução inadequada para credenciais: senhas não devem ser armazenadas como números nem ter unicidade entre usuários; o padrão atual delega autenticação ao Supabase Auth. A tabela IMAGEM não tinha chave primária declarada nem vínculo explícito com as demais entidades. A tabela associativa JARDIM_PANC não declarava chave primária composta, permitindo potencialmente duplicações. Esses aspectos são apresentados como oportunidades de evolução do modelo original, não como defeitos comprovados em produção.

4. Migração para PostgreSQL e Supabase

No Supabase, o banco PostgreSQL já é provisionado no projeto; por isso, o script de implantação não executa CREATE DATABASE, USE ou DROP DATABASE. A autenticação passa a utilizar auth.users e UUIDs. As tabelas de domínio ficam em public, com vínculos referenciais e políticas de acesso por linha. IDs de registros de domínio são gerados por identidade BIGINT quando apropriado; datas de criação utilizam TIMESTAMPTZ e NOW().

Antes (MySQL)	Depois (Supabase)	Evolução
USUARIO	auth.users + public.perfis	Separação de autenticação e perfil
PANC	public.pancs	Slug único e campos educativos adicionais
JARDIM + JARDIM_PANC	public.jardim	Associação direta por chave composta (user_id, panc_id)
IDENTIFICACAO	public.identificacoes	UUID de usuário, data/hora integradas, confiabilidade numérica
IMAGEM	public.imagens	Vínculos referenciais e identificador automático
MENSAGEM	public.mensagens	Pergunta/resposta e usuário autenticado
Sem equivalência direta	public.imagens_treinamento	Fila de imagens, autorização e revisão
Sem equivalência direta	public.admins	Controle de privilégios administrativos

5. Arquitetura atual e relacionamentos

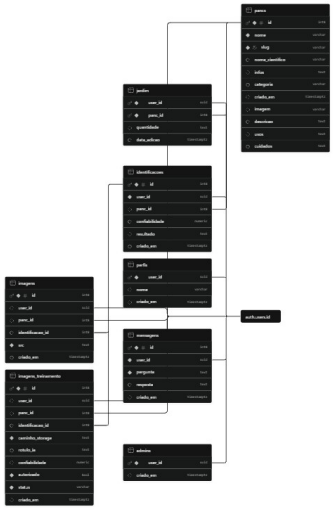


Figura — Schema Visualizer do banco Growly (captura fornecida pela equipe).

A captura do Table Editor confirma oito tabelas próprias no esquema public. A referência a auth.users é externa ao domínio das tabelas próprias: trata-se da tabela gerenciada pelo serviço de autenticação. O Storage utiliza recursos próprios, incluindo storage.objects, que também não deve ser contado como tabela de domínio do Growly.

6. Dicionário de dados detalhado

6.1 public.perfis

Campo	Tipo	Regra / restrição	Descrição
user_id	UUID	PK; FK auth.users(id); ON DELETE CASCADE	Identificador do usuário
nome	VARCHAR(150)	Opcional	Nome exibido
criado_em	TIMESTAMPTZ	DEFAULT NOW()	Criação do perfil

Fonte: scripts SQL apresentados pela equipe. O dicionário descreve a definição declarada; alterações manuais posteriores só são confirmadas quando visíveis nas capturas ou em SQL adicional.

6.2 public.pancs

Campo	Tipo	Regra / restrição	Descrição
id	BIGINT	PK; IDENTITY	Identificador da planta
nome	VARCHAR(120)	NOT NULL	Nome popular
slug	VARCHAR(120)	UNIQUE; NOT NULL	Identificador textual estável

nome_cientifico	VARCHAR(180)	Opcional	Nome científico
infos	TEXT	Opcional	Informações gerais
categoria	VARCHAR(100)	Opcional	Classificação
criado_em	TIMESTAMPTZ	DEFAULT NOW()	Criação do registro
imagem	VARCHAR(120)	Adicionado por ALTER TABLE	Chave de imagem do frontend
descricao	TEXT	Adicionado por ALTER TABLE	Descrição educativa
usos	TEXT	Adicionado por ALTER TABLE	Usos registrados
cuidados	TEXT	Adicionado por ALTER TABLE	Cuidados de utilização

Fonte: scripts SQL apresentados pela equipe. O dicionário descreve a definição declarada; alterações manuais posteriores só são confirmadas quando visíveis nas capturas ou em SQL adicional.

6.3 public.jardim

Campo	Tipo	Regra / restrição	Descrição
user_id	UUID	PK composta; FK auth.users(id); CASCADE	Proprietário
panc_id	BIGINT	PK composta; FK pancs(id); CASCADE	Planta salva
quantidade	TEXT	Opcional	Quantidade informada
data_adicao	TIMESTAMPTZ	DEFAULT NOW()	Inclusão no jardim

Fonte: scripts SQL apresentados pela equipe. O dicionário descreve a definição declarada; alterações manuais posteriores só são confirmadas quando visíveis nas capturas ou em SQL adicional.

6.4 public.identificacoes

Campo	Tipo	Regra / restrição	Descrição
id	BIGINT	PK; IDENTITY	Identificador
user_id	UUID	NOT NULL; FK auth.users(id); CASCADE	Usuário
panc_id	BIGINT	FK pancs(id); SET NULL	Planta associada
confiabilidade	NUMERIC(5,2)	Opcional	Indicador de confiança
resultado	TEXT	Opcional	Resultado textual
criado_em	TIMESTAMPTZ	DEFAULT NOW()	Data e hora

Fonte: scripts SQL apresentados pela equipe. O dicionário descreve a definição declarada; alterações manuais posteriores só são confirmadas quando visíveis nas capturas ou em SQL adicional.

6.5 public.imagens

Campo	Tipo	Regra / restrição	Descrição
id	BIGINT	PK; IDENTITY	Identificador
user_id	UUID	FK auth.users(id); CASCADE	Usuário
panc_id	BIGINT	FK pancs(id); SET NULL	Planta
identificacao_id	BIGINT	FK identificacoes(id); CASCADE	Identificação associada
src	TEXT	NOT NULL	Origem da imagem
criado_em	TIMESTAMPTZ	DEFAULT NOW()	Data e hora

Fonte: scripts SQL apresentados pela equipe. O dicionário descreve a definição declarada; alterações manuais posteriores só são confirmadas quando visíveis nas capturas ou em SQL adicional.

6.6 public.mensagens

Campo	Tipo	Regra / restrição	Descrição
id	BIGINT	PK; IDENTITY	Identificador
user_id	UUID	NOT NULL; FK auth.users(id); CASCADE	Usuário
pergunta	TEXT	NOT NULL	Pergunta à Eden
resposta	TEXT	Opcional	Resposta
criado_em	TIMESTAMPTZ	DEFAULT NOW()	Data e hora

Fonte: scripts SQL apresentados pela equipe. O dicionário descreve a definição declarada; alterações manuais posteriores só são confirmadas quando visíveis nas capturas ou em SQL adicional.

6.7 public.imagens_treinamento

Campo	Tipo	Regra / restrição	Descrição
id	BIGINT	PK; IDENTITY	Identificador
user_id	UUID	FK auth.users(id); SET NULL	Usuário de origem
panc_id	BIGINT	FK pancs(id); SET NULL	Planta associada
identificacao_id	BIGINT	FK identificacoes(id); SET NULL	Identificação associada
caminho_storage	TEXT	NOT NULL	Caminho do arquivo

rotulo_ia	TEXT	Opcional	Classificação atribuída pela IA
confiabilidade	NUMERIC(5,2)	Opcional	Confiança do resultado
autorizado	BOOLEAN	NOT NULL; DEFAULT FALSE	Autorização de uso
status	VARCHAR(30)	NOT NULL; DEFAULT pendente; CHECK	pendente, aprovada ou rejeitada
criado_em	TIMESTAMPTZ	DEFAULT NOW()	Data e hora

Fonte: scripts SQL apresentados pela equipe. O dicionário descreve a definição declarada; alterações manuais posteriores só são confirmadas quando visíveis nas capturas ou em SQL adicional.

6.8 public.admins

Campo	Tipo	Regra / restrição	Descrição
user_id	UUID	PK; FK auth.users(id); CASCADE	Identificador do administrador
criado_em	TIMESTAMPTZ	DEFAULT NOW()	Registro de concessão

Fonte: scripts SQL apresentados pela equipe. O dicionário descreve a definição declarada; alterações manuais posteriores só são confirmadas quando visíveis nas capturas ou em SQL adicional.

7. Explicação técnica dos comandos SQL

CREATE TABLE

Cria tabela e define colunas, tipos e restrições. Exemplo: CREATE TABLE public.perfis (...).

IF NOT EXISTS

Evita erro de criação quando o objeto já existe; não altera uma estrutura preexistente.

ALTER TABLE

Altera definição de tabela existente, inclusive para adicionar colunas e habilitar RLS.

ADD COLUMN IF NOT EXISTS

Adiciona coluna somente se ainda não estiver presente; usado na ampliação de pancs.

PRIMARY KEY

Impõe identificação única e não nula; em jardim combina user_id e panc_id.

REFERENCES / FOREIGN KEY

Cria integridade referencial entre registros.

ON DELETE CASCADE

Apaga registros dependentes quando o registro referenciado é excluído.

ON DELETE SET NULL
Preserva o registro dependente e remove sua referência quando o pai é excluído.

GENERATED BY DEFAULT AS IDENTITY
Gera valores numéricos de identificação automaticamente, permitindo valores explícitos conforme regras do PostgreSQL.

UUID
Tipo de identificador universal usado pela autenticação.

TIMESTAMPTZ / NOW()
Armazena data e hora com semântica de fuso; NOW() fornece instante da transação.

NUMERIC(5,2)
Tipo decimal com cinco dígitos totais e duas casas decimais.

NOT NULL / UNIQUE / CHECK
Restringem valores ausentes, duplicados ou fora de uma condição declarada.

INSERT INTO ... VALUES
Insere registros, como o catálogo inicial de cinco PANCs e os administradores.

ON CONFLICT ... DO NOTHING
Evita falha por conflito com restrição única, sem atualizar o registro existente.

UPDATE ... SET ... WHERE
Altera dados existentes conforme filtro; usado para conteúdo e imagens das PANCs.

SELECT * FROM
Consulta todas as colunas dos registros visíveis ao papel executante.

CREATE OR REPLACE FUNCTION
Define ou substitui uma função de banco, como is_admin() ou criar_perfil_usuario().

LANGUAGE sql / plpgsql
Identifica a linguagem da função: SQL declarativo ou linguagem procedural PostgreSQL.

SECURITY DEFINER
Executa a função com privilégios do proprietário; exige cuidados com proprietário e search_path.

SET search_path = public
Define resolução de objetos na função; objetos sensíveis podem ser qualificados explicitamente.

RETURNS BOOLEAN / TRIGGER
Define se a função retorna um valor lógico ou participa de um gatilho.

CREATE TRIGGER / AFTER INSERT
Executa função automaticamente após inclusão de usuário em auth.users.

NEW.id / raw_user_meta_data
Acessa valores do novo registro de autenticação para criar o perfil.

ENABLE ROW LEVEL SECURITY
Habilita a avaliação de políticas por linha na tabela.

CREATE POLICY
Declara condições de autorização por operação e papel.

USING
Determina quais linhas podem ser vistas, alteradas ou excluídas conforme operação.

WITH CHECK
Verifica se a nova linha inserida/alterada atende à condição de acesso.

TO authenticated
Direciona a política ao papel de usuários autenticados.

auth.uid()
Obtém o identificador do usuário da sessão Supabase.

storage.foldername(name)
Extrai componentes do caminho de arquivo no Supabase Storage.

DROP POLICY IF EXISTS
Remove política anterior antes de recriá-la, quando previsto no script.

8. Segurança e políticas de acesso
Row Level Security (RLS) restringe as linhas acessíveis a cada papel, complementando privilégios SQL de tabela. As políticas enviadas permitem acesso do usuário aos próprios dados em perfis, jardim, identificacoes, imagens, mensagens e imagens_treinamento; o catálogo pancs possui leitura para usuários autenticados. A tabela admins permite consulta do próprio registro e a função is_admin() verifica a existência do usuário em public.admins.

Objeto	Operações documentadas	Condição principal
perfis	SELECT, UPDATE	auth.uid() = user_id
pancs	SELECT	authenticated; USING (true)
jardim	SELECT, INSERT, UPDATE, DELETE	auth.uid() = user_id
identificacoes	SELECT, INSERT, DELETE	auth.uid() = user_id
imagens	SELECT, INSERT, DELETE	auth.uid() = user_id
mensagens	SELECT, INSERT, DELETE	auth.uid() = user_id
imagens_treinamento	INSERT e SELECT do usuário; SELECT e UPDATE do admin	user_id ou public.is_admin()
admins	SELECT do próprio registro	auth.uid() = user_id

Os scripts complementares mostram também políticas administrativas de INSERT, UPDATE e DELETE no bucket imagens-treinamento e políticas de inserção e leitura da própria pasta para usuários comuns. A contagem visual no painel é de seis políticas para esse bucket; os scripts históricos contêm variantes e não devem ser somados indiscriminadamente.

8.1 Função administrativa e princípio de menor privilégio

```
CREATE OR REPLACE FUNCTION public.is_admin()
RETURNS BOOLEAN
LANGUAGE sql
SECURITY DEFINER
SET search_path = public
AS $$
  SELECT EXISTS (
    SELECT 1 FROM public.admins
    WHERE user_id = auth.uid()
  );
$$;
```

A função consulta o vínculo administrativo sem exigir que o usuário comum leia toda a tabela de administradores. Como SECURITY DEFINER amplia o contexto de privilégios, recomenda-se auditar proprietário, permissões EXECUTE e dependências. O script registra quatro UUIDs de administradores; por privacidade, seus valores completos não são reproduzidos neste relatório.

8.2 Trigger de perfil

```
CREATE TRIGGER criar_perfil_apos_cadastro
AFTER INSERT ON auth.users
FOR EACH ROW
EXECUTE FUNCTION public.criar_perfil_usuario();
```

A função associada insere em public.perfis o id e o nome fornecido nos metadados de autenticação; ON CONFLICT (user_id) DO NOTHING evita duplicidade. A execução depende da configuração e dos privilégios da função no ambiente.

8.3 Storage

Bucket	Políticas na captura	Limite por arquivo	MIME permitido
avatares	4	5 MB	image/jpeg, image/png, image/webp
imagens-treinamento	6	5 MB	image/jpeg

Os scripts de avatares vinculam o primeiro segmento do caminho ao auth.uid()::text, restringindo inserção, leitura, atualização e exclusão à pasta própria. Os scripts de imagens-treinamento incluem acesso à pasta do usuário e administração por public.is_admin(). A captura de Storage confirma nomes, contagens e limites, mas não demonstra sozinha o caráter público/privado de cada bucket nem a eficácia das políticas.

9. Fluxos de dados e integração

9.1 Cadastro e perfil

O Supabase Auth registra a conta em auth.users. Um gatilho AFTER INSERT chama criar_perfil_usuario(), que cria o registro correspondente em public.perfis. O acesso ao perfil é delimitado por RLS.

9.2 Catálogo e jardim

O aplicativo consulta pancs e apresenta conteúdo por slug. O campo imagem é preenchido com chaves compatíveis com o mapeamento PLANT_IMAGES do frontend, conforme comentário do script. A tabela jardim usa chave composta, impedindo que a mesma combinação de usuário e PANC seja duplicada.

9.3 Identificações e imagens

identificacoes registra o resultado e a confiabilidade. imagens referencia usuários, PANCs e identificações, enquanto imagens_treinamento possui caminho de arquivo, rótulo, autorização e estado de revisão. As imagens de treinamento não devem ser confundidas com os próprios pesos ou arquivos do modelo de IA.

9.4 Eden

mensagens registra pergunta e resposta, com vínculo ao usuário. A integração com a assistente e o processamento externo são componentes de aplicação e não estão integralmente descritos pelo SQL apresentado. Alterações recentes de autenticação da função de backend podem ser mencionadas como contexto técnico, mas sua eficácia exige evidência própria.

10. Análise crítica e recomendações

Consistência entre imagem e identificação

As FKs garantem existência da identificação, mas não impõem que user_id da imagem corresponda ao proprietário da identificação. Recomenda-se validação adicional no backend ou mecanismo relacional apropriado.

Consentimento para treinamento

O campo autorizado começa como FALSE. Recomenda-se que a seleção de imagens para treinamento exija autorização afirmativa, aprovação quando aplicável e regras claras de retirada.

Confiança do modelo

NUMERIC(5,2) não restringe automaticamente valores a 0–100. Se esse for o intervalo esperado, adicionar CHECK correspondente após verificar a convenção utilizada.

Nomes e política de atualização

A política de UPDATE de perfis deve ser combinada com permissões de coluna para impedir mudanças indevidas em campos de controle; o SQL de RLS sozinho não prova os GRANTs.

Consolidação de migrações

Separar scripts históricos de um esquema de implantação reproduzível; CREATE POLICY repetido pode falhar se política de mesmo nome já existir.

Integridade do catálogo

Verificar alinhamento de slug, chaves de imagem e espécies no frontend; o SQL mostra cinco plantas iniciais.

Privacidade de capturas

Evitar publicar UUIDs integrais, nomes e emails reais de usuários em anexos acadêmicos; utilizar recortes ou anonimização.

Testes documentais

Criar matriz com usuário A, usuário B, administrador e anônimo, registrando resultados de SELECT, INSERT, UPDATE, DELETE e acesso ao Storage.

11. Matriz de testes proposta — ainda não executada neste relatório

Cenário	Resultado esperado
Usuário A consulta jardim de B	Nenhuma linha de B deve ser retornada
Usuário A insere jardim para B	Operação negada
Usuário autenticado consulta pancs	Leitura permitida
Usuário comum consulta admins de outro usuário	Nenhuma linha retornada
Admin consulta imagens_treinamento	Leitura permitida conforme políticas vigentes
Usuário envia avatar para pasta de outro UUID	Operação negada
Cadastro cria perfil	Registro criado por trigger
Imagem vinculada a identificação de terceiro	Verificar bloqueio adicional; FK isolada não basta

12. Glossário técnico e siglas

Termo	Definição
ABNT	Associação Brasileira de Normas Técnicas
API	Application Programming Interface — interface de programação de aplicações
Auth	Módulo de autenticação
BIGINT	Número inteiro de 64 bits
CRUD	Create, Read, Update, Delete — criar, ler, atualizar e excluir
DDL	Data Definition Language — linguagem de

	definição de dados
DML	Data Manipulation Language — linguagem de manipulação de dados
Eden	Assistente do Growly
FK	Foreign Key — chave estrangeira
IA	Inteligência Artificial
IDENTITY	Mecanismo de geração automática de identificadores
MIME	Multipurpose Internet Mail Extensions — identificação de tipo de conteúdo
MySQL	Sistema gerenciador de banco de dados relacional
PANC	Planta Alimentícia Não Convencional
PK	Primary Key — chave primária
PostgreSQL	Sistema gerenciador de banco de dados relacional objeto-relacional
RLS	Row Level Security — segurança em nível de linha
SQL	Structured Query Language — linguagem de consulta estruturada
Storage	Armazenamento de arquivos
Supabase	Plataforma de backend baseada em PostgreSQL e serviços associados
TCC	Trabalho de Conclusão de Curso
TIMESTAMPTZ	Timestamp with time zone — data/hora com informação de fuso
Trigger	Gatilho de banco acionado por evento
UUID	Universally Unique Identifier — identificador universalmente único
VARCHAR	Campo textual de tamanho variável

13. Conclusão

O banco de dados do Growly evoluiu de um protótipo MySQL com sete tabelas para uma arquitetura PostgreSQL/Supabase com oito tabelas próprias, autenticação externa ao esquema de domínio, regras de integridade, mecanismos de controle por linha e armazenamento de arquivos. A evolução responde às necessidades funcionais do aplicativo e representa amadurecimento da modelagem e das práticas de segurança. As evidências recebidas sustentam a existência da estrutura e das políticas declaradas, mas não substituem uma auditoria de permissões e testes operacionais documentados.

Referências documentais e anexos

- GROWLY DATABASE.sql — protótipo original em MySQL, fornecido pela equipe.
- Diário de Bordo (Oficial) (1).docx — registros de fevereiro a junho de 2026, com entrada posterior de julho ainda em branco.
- Dez materiais SQL de PostgreSQL/Supabase fornecidos pela equipe — definição, catálogo, RLS, Storage, administradores e imagens de treinamento.
- Capturas de tela do Supabase Table Editor, Schema Visualizer e Storage — outubro de 2026.
- Diários semanais do Growly — conjunto de PDFs disponibilizado pela equipe, junho–agosto de 2026.

Anexo A — Script histórico MySQL

```
drop database if exists growly;
create database growly;
use growly;

CREATE TABLE USUARIO (
        id int unique primary key,
        nome varchar(400),
        email text,
        ursn varchar(20),
        senha int(8) unique
);

CREATE TABLE PANC(
        id_panc int unique primary key,
        n_PANC varchar(111),
        infos text,
        categ varchar(100)
);

CREATE TABLE IDENTIFICACAO(
        id_iden int unique primary key,

        id int,
        id_panc int,
        foreign key (id) references USUARIO(id),
        foreign key (id_panc) references PANC(id_panc),

        dat date,
        hora time,
        padr text,
        p_d_conf int
);

CREATE TABLE IMAGEM (
        id_img int unique,
   src text
);
```

```
CREATE TABLE JARDIM (
	id_jardim int unique primary key,

	id int,
	foreign key (id) references USUARIO(id),

	resg text
);

CREATE TABLE JARDIM_PANC(
	id_jardim int,
	id_panc int,
	foreign key (id_jardim) references JARDIM(id_jardim),
	foreign key (id_panc) references PANC(id_panc),

	quantidade text,
	dat_add date
);

CREATE TABLE MENSAGEM(
	id_msg int unique primary key,

	id int,
	id_iden int,
	foreign key (id) references USUARIO(id),
	foreign key (id_iden) references IDENTIFICACAO(id_iden),

	txt text,
	resp text,
	dat date,
	hora time
);
```

Anexo B — Exemplos representativos de políticas RLS

```
ALTER TABLE public.jardim ENABLE ROW LEVEL SECURITY;
CREATE POLICY "Usuario pode visualizar proprio jardim"
ON public.jardim FOR SELECT TO authenticated
USING (auth.uid() = user_id);

CREATE POLICY "Usuario pode adicionar ao proprio jardim"
ON public.jardim FOR INSERT TO authenticated
WITH CHECK (auth.uid() = user_id);
```

Observação editorial: os dez scripts completos permanecem como fontes de referência da equipe. Este anexo traz exemplos e o SQL original integral; uma edição posterior pode incorporar cada arquivo de migração integralmente, com identificação de versão e ordem de execução.